

API Design, Evolution & Authorization Cheatsheet

Lessons: [41 — API Design & Evolution](#) · [43 — Authorization, RBAC & Multi-Tenancy](#)

Examples: [41](#)

Covers: resource design, compatible change, pagination, idempotency, `problem+json`, RBAC, tenant scoping

Legend: `[*]` = concept or standard the lessons have not covered yet

RESOURCE DESIGN

nouns, not verbs	POST /orders, not POST /createOrder
plural collections	/orders, /orders/{id}, /orders/{id}/items
GET	safe and idempotent; never changes state
POST	create, or "run this action"; NOT idempotent by default
PUT	full replace, idempotent
PATCH	partial update (JSON Merge Patch is the simple choice)
DELETE	idempotent: deleting twice is still 204
nest one level deep	/orders/{id}/items, then stop
filter with the query string	/orders?status=open&sort=-created_at
sub-resources for actions	POST /orders/{id}/cancel when it isn't really CRUD
ids	opaque strings; UUID/ULID, never a leaking sequence

COMPATIBLE CHANGE (the rules)

SAFE	add an optional request field add a response field add a new endpoint add a new enum value ONLY if clients tolerate unknowns
BREAKING	remove or rename anything change a type (int -> string, scalar -> object) make an optional field required tighten validation change a status code or an error shape change the default of anything
tolerant reader	ignore unknown fields on the way in
never reuse a name	with a new meaning
deprecate, then remove	Deprecation/Sunset headers, docs, a migration window
version when you must	/v1/ in the path is the simplest thing that works header/media-type versioning is purer and less used
one version at a time	supporting three versions costs three test suites

PAGINATION

offset/limit	?page=3&per_page=50 – simple, and it degrades: deep pages get slow, and rows shift under the reader
cursor (keyset)	?cursor=<opaque>&limit=50 – THE production answer WHERE (created_at, id) < (\$1, \$2) ORDER BY created_at DESC, id DESC LIMIT \$3 the cursor encodes the last row's sort key; base64 it so it stays opaque stable under inserts, and O(1) at any depth
response shape	{ "data": [...], "next_cursor": "..." }
always cap the limit	and document the default and the max
total counts are expensive	make them optional, or drop them

ERRORS: RFC 9457 problem+json

Content-Type: application/problem+json

```
{
  "type": "https://api.example.com/errors/insufficient-funds",    a stable URI
  "title": "Insufficient funds",                                   short, human, does not change per instance
  "status": 402,                                                  matches the HTTP status
  "detail": "Balance 30 is below 50",                             this occurrence
  "instance": "/accounts/42/transfers/9", the specific resource
  "errors": [ {"field": "amount", "message": "must be positive"} ] extensions
}
```

one error shape for the whole API clients parse it once

machine-readable "type" never make clients regex the message

field-level errors for 422 the form can highlight the right input

never leak internals no stack traces, no SQL, no table names

map domain errors centrally one place turns ErrNotFound into 404

IDEMPOTENCY

Idempotency-Key: <uuid> client-generated, per logical operation

store (key, request_hash, response) for 24h

same key + same body return the STORED response, don't re-execute

same key + different body 409 Conflict

in-flight 409, or block briefly

apply it to POST that creates or charges – anything with money

(and make the DB write itself idempotent: a unique constraint on the key)

API HYGIENE

Content-Type: application/json and check it on input (415 otherwise)

UTC + RFC 3339 timestamps "2026-08-30T12:00:00Z", always

money as integer minor units cents, never a float

enums as strings "pending", not 3

consistent casing snake_case or camelCase – pick one, forever

null vs absent vs empty decide and document; PATCH depends on it

ETag / If-None-Match [*] conditional GET; 304 saves bandwidth

If-Match on writes [*] optimistic concurrency at the HTTP layer

OpenAPI spec generated from or checked against the code

AUTHN vs AUTHZ

authentication WHO are you -> 401 Unauthorized

authorization MAY you do this -> 403 Forbidden

401 sends WWW-Authenticate and means "try again with credentials"

403 means "don't bother" the same request will never succeed

404 instead of 403 [*] when the existence of the resource is itself secret

authn is lesson 56 this sheet is the authz half

RBAC

```
users -> roles -> rights      two pivot tables, one direction of travel
  users, roles, rights
  user_roles(user_id, role_id)
  role_rights(role_id, right_id)
rights are FINE-GRAINED      "case.view", "case.assign", "invoice.refund"
roles are BUNDLES            admin, agent, auditor – they change; rights don't
check RIGHTS, never roles    if role == "admin" is the bug that outlives the codebase
RequireRight("case.view")    middleware between the router and the handler
  resolve identity from ctx -> load rights -> 403 if missing
cache the right set          per user, short TTL or invalidated on role change
identity in the context      set once by auth middleware, read everywhere
  type userKey struct{}; ctx = context.WithValue(ctx, userKey{}, u)
ABAC / ownership             "may edit THIS case" needs the row, not just the right
                             – check the right first, then the ownership rule
```

MULTI-TENANCY

```
the model                    every tenant-owned row carries tenant_id
scope EVERY query            WHERE tenant_id = $1 – no exceptions, ever
get the tenant from the TOKEN never from a request body or a query param
enforce it in the repository the layer where forgetting is impossible
  func (r *Repo) ByID(ctx, id) – reads the tenant from ctx itself
row-level security          [*] Postgres RLS as a second line of defence
separate schemas / DBs      [*] stronger isolation, heavier operations
composite index              (tenant_id, ...) – tenant_id first, always
the test that matters        tenant A must get 404 for tenant B's id
```

OVERPOSTING (mass assignment)

```
the attack                    POST body includes "role":"admin" or "tenant_id":"other"
                             and it lands straight in the model
the defense                    a request DTO with ONLY the client-settable fields
                             then STAMP the trusted fields from the context:
  o := domain.Order{TenantID: auth.TenantID(ctx), CreatedBy: auth.UserID(ctx)}
  o.Amount = in.Amount          only what the client may set
DisallowUnknownFields()        reject the extra key loudly instead of ignoring it
never json.Unmarshal into the domain model
never bind an update straight onto the loaded row
```

TRAPS & MEMORIZE

verbs in URLs	/getUser – the API is not an RPC namespace
breaking change without a version	every client breaks at once, silently
OFFSET pagination at scale	slow and unstable; use keyset
error messages as the API	clients WILL parse them; give them a "type"
leaking internal errors	stack traces and SQL in a 500 body
201 without a Location	the client can't find what it created
floats for money	0.1 + 0.2 in someone's invoice
checking roles not rights	every new role becomes a code change
tenant_id from the request	the whole isolation model, defeated
forgetting one WHERE clause	a cross-tenant data leak in one endpoint
authorization in the handler	30 handlers, 30 chances to forget – do it in middleware and again in the repository
403 that leaks existence	when the resource itself is confidential, 404
idempotency without storage	"idempotent" that re-executes on retry